PropOS 开发与 UI 设计同步指南

> 版本: v1.0
>
> 日期: 2026-04-04
>
> 适用项目: PropOS Phase 1
>
> 适用角色: 开发者 + UI/UX 设计师

目录

1. 核心理念
    - 1.1 问题：传统流程的痛点
    - 1.2 PropOS 方案："骨架先行，并行打磨"
2. 整体工作流
3. 分阶段执行计划
    - Phase A：骨架生成（Day 0~2）
    - Phase B：对齐日（Day 3）
    - Phase C：双线并行（Day 4~10）
    - Phase D：视觉接入（Day 11~14）
4. 角色职责分工
    - 4.1 开发者职责
    - 4.2 设计师职责
    - 4.3 禁止事项
5. 设计师输入物清单
    - 5.1 截图包（按页面编号）
    - 5.2 业务色彩语义定义
    - 5.3 数据边界速查表
6. Design Token 规范
    - 6.1 颜色 Token
    - 6.2 字体 Token
    - 6.3 间距 Token
    - 6.4 圆角 Token
7. Flutter ThemeData 对接规范
8. 页面清单与优先级
    - P0：第一周必须完成（验收基础）
    - P1：第二周完成
    - P2：第三周完成

9. 协作检查清单
9.1 开始骨架开发前（Day 0）
9.2 对齐日前（Day 3）
9.3 Design Token 接入前（Day 11）
9.4 视觉验收（Day 14）
附录：快速参考
状态色彩速查
Mock 数据替换路径

---

1. 核心理念

1.1 问题：传统流程的痛点

传统"设计先行"流程：

```
① 需求文档 → ② 设计师出 Figma 稿 → ③ 开发实现 → ④ 返工对齐
```

痛点：

- 设计师对数据密度没有感知（"这个列表一行到底几个字段？"）
- 开发实现后发现布局问题，两端同时返工
- 设计与代码长期存在偏差，无专人维护 Design Token

1.2 PropOS 方案："骨架先行，并行打磨"

```
① 开发生成 Flutter 骨架（MockData）
            ↓
② 模拟器截图 + 录屏 → 设计师看真实数据密度
            ↓
③ 开发 + 设计双线并行推进
            ↓
④ 设计师输出 Design Token → 开发写入 ThemeData
            ↓
⑤ 视觉精修：按 Figma 稿调整间距、色彩、字体
```

核心原则：

- **骨架即生产代码**，不存在"原型→推倒重来"
- **MockData 可随时替换为真实 API**，不影响页面逻辑
- **Design Token 驱动**，设计更改只改 ThemeData，不改 Widget

---

2. 整体工作流

```
┌──────────────────────────────────────────────────────────────┐
│  第 0 天：启动准备                                            │
│  开发者：环境搭建 + pubspec.yaml + 路由骨架                   │
│  设计师：阅读 PRD.md + ARCH.md §4（路由结构）+ 竞品参考       │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 1~2 天：骨架生成                                          │
│  开发者：生成全部页面 Widget 骨架 + MockData                  │
│  设计师：整理色彩参考、字体偏好、业务风格定义（科技感/严肃感等）│
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 3 天：对齐日                                              │
│  开发者：flutter run + 录制每个 Tab 的操作录屏（共约 5 段）   │
│  设计师：看录屏，标注疑问："这里要不要分页？""这个状态色块太小了"│
│  联合：30 分钟站会，统一疑问 → 形成设计约束文档（本文附录）   │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 4~10 天：双线并行                                         │
│  开发者：继续完善业务逻辑、状态管理、错误处理                 │
│  设计师：基于截图在 Figma 做视觉设计，输出 Design Token 表    │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 11~14 天：视觉接入                                        │
│  开发者：将 Design Token 写入 ThemeData，按 Figma 调整间距细节 │
│  设计师：验收视觉效果，输出标注修正（Redline）                │
└──────────────────────────────────────────────────────────────┘
```

3. 分阶段执行计划

Phase A：骨架生成（Day 0~2）

开发者交付物：

文件	说明
`pubspec.yaml`	依赖声明（go*router*、*flutter*bloc、fl_chart 等）
`lib/main.dart`	App 入口，MockLoginState 注入
`lib/router/app_router.dart`	完整路由树（35 个页面全部注册）
`lib/mock/mock_*.dart`	各模块 MockData（覆盖所有页面所需字段）
`lib/shared/theme/app_theme.dart`	临时占位 ThemeData（Material 3 默认色）

所有 `*Page` Widget 文件	骨架页面，含正确的布局结构和 MockData 展示

MockData 覆盖范围：

- `MockDashboardData`：NOI、EGI、OpEx、出租率、WALE（按三业态）
- `MockBuildingData`：3 栋楼 × 多楼层 × 单元列表（含状态分布）
- `MockContractData`：10 份合同（覆盖全部状态机状态）
- `MockInvoiceData`：20 条账单（覆盖 pending/paid/overdue）
- `MockWorkOrderData`：8 条工单（覆盖全部状态）
- `MockSubleaseData`：二房东视角的子租赁列表

Phase B：对齐日（Day 3）

开发者输出（给设计师）：

1. 每个主 Tab 页面截图（iPhone 14 Pro 尺寸）
2. 核心流程操作录屏：
    - 登录 → Dashboard 总览
    - 资产 → 楼栋 → 楼层热区图 → 单元详情
    - 合同列表 → 合同详情 → 租金递增配置
    - 财务 → 账单列表 → 核销操作
    - 工单提报全流程
1. 本文 §8 页面清单（标注"高优"页面）

会议议程（30 分钟）：

时间	内容
0~10 分钟	开发者演示骨架 App 操作流程
10~20 分钟	设计师提问：数据密度、交互边界、色块语义
20~30 分钟	联合确认：状态色彩语义、字体大小底线、卡片信息层级

Phase C：双线并行（Day 4~10）

开发者任务：

- 完善 BLoC 状态管理框架（Event / State 定义）
- 接入 go_router 路由守卫（RBAC 角色判断）
- 完善表单页面的验证逻辑
- 处理空状态、加载状态、错误状态三种 UI 状态

设计师任务：

优先级	Figma 页面	输出要求
P0	Dashboard 总览	完整高保真，含数据可视化组件样式
P0	楼层平面图热区	四种状态色定义 + 悬浮弹窗样式
P1	合同详情 + 递增配置器	复杂表单布局
P1	账单列表 + 核销弹窗	列表行设计
P2	工单流程（提报/审核/完工）	移动优先布局
P3	二房东填报 Portal	独立简洁风格，与主 App 视觉上有轻微差异

Phase D：视觉接入（Day 11~14）

1. 设计师提交 Design Token 表（见 §6）
2. 开发者将 Token 写入 `lib/shared/theme/app_theme.dart`
3. 逐页核对 Figma 标注，调整 `padding` / `borderRadius` / `fontSize`
4. 设计师在模拟器上验收，提交 Redline 修正清单
5. 开发者修正后，进入功能测试阶段

---

4. 角色职责分工

4.1 开发者职责

职责	说明
主导目录结构	严格按 ARCH.md §2 建立，不允许设计师决定文件命名
维护 MockData	MockData 字段须与真实 API 响应结构一致，便于后续替换
定义组件接口	在生成骨架时确定 Widget 参数签名，设计师不得要求修改参数名
接入 Design Token	将设计师的 Token 表翻译为 Dart 代码，设计师不直接改代码
性能基线	列表页使用 `ListView.builder`，图片使用 `cached_network_image`

4.2 设计师职责

职责	说明
了解数据边界	必须知道每个字段的最大长度（如合同编号最长 100 字符）
respect 业务状态	状态色块（🟢🟡🔴⚪）是业务约束，不可仅凭美观更改颜色语义
输出 Design Token	以表格形式输出（见 §6），不直接修改代码
标注单位	所有间距以 dp（逻辑像素）标注，不使用 px
验收使用真机/模拟器	不以 Figma 预览验收，必须在 Flutter 模拟器上确认

4.3 禁止事项

禁止行为	原因
设计师直接修改 Flutter 代码	破坏代码分层约定
开发者绕过 Figma 自行"美化"	产生设计漂移，难以维护
设计师更改状态色彩语义	`expiring_soon` 必须是黄色系，这是业务约束
使用 Figma 截图验收替代模拟器	Figma 不模拟真实屏幕密度和字体渲染

---

5. 设计师输入物清单

开发者在 Day 3 交给设计师的标准输入包：

5.1 截图包（按页面编号）

```
screenshots/
├── 01_login.png
├── 02_dashboard.png
├── 03_dashboard_noi_detail.png
├── 04_dashboard_wale_detail.png
├── 05_dashboard_kpi.png
├── 06_assets_overview.png
├── 07_building_detail.png
├── 08_floor_map.png              ← 热区色块核心页
├── 09_unit_detail.png
├── 10_contract_list.png
├── 11_contract_detail.png
├── 12_escalation_config.png
```

```
├── 13_finance_overview.png
├── 14_invoice_list.png
├── 15_workorder_list.png
├── 16_workorder_new.png
├── 17_workorder_detail.png
├── 18_sublease_portal.png        ← 二房东独立入口
└── 19_settings.png
```

5.2 业务色彩语义定义

语义	含义	说明
`status.leased`	🟢 已租	必须是绿色系
`status.expiring_soon`	🟡 即将到期（≤90天）	必须是黄/橙色系
`status.vacant`	🔴 空置	必须是红色系
`status.non_leasable`	⚪ 非可租	中性灰色
`invoice.overdue`	🔴 逾期账单	与空置同语义色系
`invoice.paid`	🟢 已核销	与已租同语义色系
`alert.warning`	🟡 预警通知	与即将到期同语义色系

5.3 数据边界速查表

字段	最大长度	说明
楼栋名称	10 字符	如"A座写字楼"
合同编号	100 字符	系统生成，不会换行
租客名称	200 字符	企业名可能很长，须处理溢出
单元编号	50 字符	如"A-1501"
工单描述	无限制（TEXT）	列表页需要截断显示
金额	最大 12 位整数 + 2 位小数	须考虑对齐方式（右对齐）

---

6. Design Token 规范

设计师完成 Figma 后，以如下格式提交 Design Token 表格（Excel 或 Markdown）：

6.1 颜色 Token

Token 名称	Hex 值	使用场景
`color.primary`	`#1976D2`	主品牌色、按钮、链接
`color.primary.container`	`#BBDEFB`	主色容器背景
`color.secondary`	`#388E3C`	成功/已租状态
`color.error`	`#D32F2F`	错误/空置/逾期
`color.warning`	`#F57C00`	预警/即将到期
`color.surface`	`#FAFAFA`	卡片背景
`color.background`	`#F5F5F5`	页面背景
`color.on.surface`	`#212121`	主文字色
`color.on.surface.variant`	`#757575`	次要文字色

> 注意：Token 名称由开发者定义，设计师填入对应的 Hex 值。Token 名称不可更改。

6.2 字体 Token

Token 名称	字号	字重	用途
`text.display.large`	32sp	Bold	大看板数字（NOI、出租率）
`text.display.medium`	24sp	Bold	次级看板数字
`text.headline.large`	20sp	SemiBold	页面标题
`text.headline.medium`	16sp	SemiBold	卡片标题
`text.body.large`	16sp	Regular	正文
`text.body.medium`	14sp	Regular	列表行内容
`text.label.large`	14sp	Medium	按钮文字
`text.label.small`	11sp	Regular	徽标、状态标签

6.3 间距 Token

Token 名称	值（dp）	使用场景
`spacing.xs`	4	图标与文字间距
`spacing.sm`	8	组件内部间距
`spacing.md`	16	卡片 padding、列表行间距
`spacing.lg`	24	页面水平边距
`spacing.xl`	32	页面顶部间距

6.4 圆角 Token

Token 名称	值（dp）	使用场景
`radius.sm`	4	小标签、徽标
`radius.md`	8	输入框
`radius.lg`	12	卡片
`radius.xl`	16	底部弹窗、对话框

---

7. Flutter ThemeData 对接规范

开发者依据 §6 的 Token 表，填入以下模板：

```
// lib/shared/theme/app_theme.dart

import 'package:flutter/material.dart';

// ── Design Token 常量 ──────────────────────────────────────────
// 由设计师提供 Hex 值后填入，命名不可更改
abstract class AppColors {
  static const primary           = Color(0xFF1976D2);  // ← 设计师填写
  static const primaryContainer  = Color(0xFFBBDEFB);
  static const secondary         = Color(0xFF388E3C);  // 已租/成功
  static const error             = Color(0xFFD32F2F);  // 空置/逾期
  static const warning           = Color(0xFFF57C00);  // 预警/即将到期
  static const surface           = Color(0xFFFAFAFA);
  static const background        = Color(0xFFF5F5F5);
  static const onSurface         = Color(0xFF212121);
  static const onSurfaceVariant  = Color(0xFF757575);

  // 业务语义色（固定，设计师不可更改）
  static const unitLeased        = secondary;
```

```
  static const unitExpiring      = warning;
  static const unitVacant        = error;
  static const unitNonLeasable   = Color(0xFF9E9E9E);
}

abstract class AppTextStyles {
  static const displayLarge  = TextStyle(fontSize: 32, fontWeight:
FontWeight.bold);
  static const displayMedium = TextStyle(fontSize: 24, fontWeight:
FontWeight.bold);
  static const headlineLarge = TextStyle(fontSize: 20, fontWeight:
FontWeight.w600);
  static const headlineMedium= TextStyle(fontSize: 16, fontWeight:
FontWeight.w600);
  static const bodyLarge     = TextStyle(fontSize: 16, fontWeight:
FontWeight.normal);
  static const bodyMedium    = TextStyle(fontSize: 14, fontWeight:
FontWeight.normal);
  static const labelLarge    = TextStyle(fontSize: 14, fontWeight:
FontWeight.w500);
  static const labelSmall    = TextStyle(fontSize: 11, fontWeight:
FontWeight.normal);
}

abstract class AppSpacing {
  static const xs = 4.0;
  static const sm = 8.0;
  static const md = 16.0;
  static const lg = 24.0;
  static const xl = 32.0;
}

abstract class AppRadius {
  static const sm = Radius.circular(4);
  static const md = Radius.circular(8);
  static const lg = Radius.circular(12);
  static const xl = Radius.circular(16);
}

// ── ThemeData 组装 ──────────────────────────────────────────
ThemeData buildAppTheme() {
  return ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.light(
      primary:          AppColors.primary,
      primaryContainer: AppColors.primaryContainer,
      secondary:        AppColors.secondary,
      error:            AppColors.error,
      surface:          AppColors.surface,
      onSurface:        AppColors.onSurface,
    ),
    scaffoldBackgroundColor: AppColors.background,
    textTheme: const TextTheme(
      displayLarge:   AppTextStyles.displayLarge,
      displayMedium:  AppTextStyles.displayMedium,
```

```
      headlineLarge:  AppTextStyles.headlineLarge,
      headlineMedium: AppTextStyles.headlineMedium,
      bodyLarge:      AppTextStyles.bodyLarge,
      bodyMedium:     AppTextStyles.bodyMedium,
      labelLarge:     AppTextStyles.labelLarge,
      labelSmall:     AppTextStyles.labelSmall,
    ),
    cardTheme: CardTheme(
      color: AppColors.surface,
      elevation: 1,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.all(AppRadius.lg),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.all(AppRadius.md),
      ),
      contentPadding: EdgeInsets.symmetric(
        horizontal: AppSpacing.md,
        vertical: AppSpacing.sm,
      ),
    ),
  );
}
```

设计师只需修改`**AppColors**` 中的 **Hex** 值，90% 的视觉变更会自动传导到全 App。

---

8. 页面清单与优先级

P0：第一周必须完成（验收基础）

编号	页面	路由	设计复杂度
01	登录页	`/login`	★☆☆
02	Dashboard 总览	`/dashboard`	★★★
03	资产概览	`/assets`	★★☆
04	楼层平面图（热区）	`/assets/building/:id/floor/:id`	★★★
05	单元详情	`/assets/.../unit/:id`	★★☆
06	合同列表	`/contracts`	★☆☆
07	合同详情	`/contracts/:id`	★★★

P1：第二周完成

编号	页面	路由	设计复杂度
08	财务概览（NOI 看板）	`/finance`	★★★
09	账单列表	`/finance/invoices`	★★☆
10	工单列表	`/workorders`	★★☆
11	工单提报	`/workorders/new`	★★☆
12	KPI 看板	`/dashboard/kpi`	★★★
13	租金递增配置器	`/contracts/:id/escalation`	★★★

P2：第三周完成

编号	页面	路由	设计复杂度
14	二房东填报 Portal	`/sublease-portal`	★★☆
15	工单详情/审核/完工	`/workorders/:id/*`	★☆☆
16	租客管理	`/tenants`	★☆☆
17	用户管理/系统设置	`/settings/*`	★☆☆
18	支出录入	`/finance/expenses/new`	★☆☆

---

9. 协作检查清单

9.1 开始骨架开发前（Day 0）

- [ ] `flutter --version` 确认 3.x
- [ ] `pubspec.yaml` 依赖包版本锁定（go*router*、*flutter*bloc、fl_chart）
- [ ] MockData 字段名与 ARCH.md §3 数据库字段名保持一致
- [ ] `AppColors` 占位色已写入，骨架阶段用 Material 默认色

9.2 对齐日前（Day 3）

- [ ] 所有 P0 页面可在模拟器中完整浏览（无崩溃）
- [ ] 截图包已导出（19 张）
- [ ] 5 段操作录屏已录制（建议用 QuickTime 录屏）
- [ ] 本文 §5.3 数据边界速查表已交给设计师

9.3 Design Token 接入前（Day 11）

- [ ] 设计师已提交完整 Token 表（颜色/字体/间距/圆角）
- [ ] P0 页面 Figma 稿已完成（含标注）
- [ ] 开发者已将 Token 填入 `app_theme.dart`
- [ ] 业务语义色（unitLeased/unitExpiring/unitVacant）未被设计师更改语义

9.4 视觉验收（Day 14）

- [ ] 设计师在 iPhone 14 Pro 模拟器验收所有 P0 页面
- [ ] 设计师在 iPad 模拟器验收 Dashboard（Web 管理后台基准）
- [ ] Redline 修正清单已提交并修复
- [ ] `flutter analyze` 无 Warning

附录：快速参考

状态色彩速查

```
Color unitStatusColor(String status) => switch (status) {
  'leased'        => AppColors.unitLeased,
  'expiring_soon' => AppColors.unitExpiring,
  'vacant'        => AppColors.unitVacant,
  'non_leasable'  => AppColors.unitNonLeasable,
  _               => AppColors.onSurfaceVariant,
};
```

Mock 数据替换路径

当后端 API 就绪时，替换方式如下：

```
// 原来（原型阶段）
final data = MockDashboardData.current();

// 替换为（接入真实 API）
final data = await context.read<DashboardBloc>().fetchSummary();
```

页面 Widget 代码**不需要任何改动**，只改数据源。

文档结束 · 如有疑问联系开发负责人